

```
In [2]: from sage.all import *
```

```
In [3]: """ Returns the volume of G """  
def vol(G):  
    V = 0  
    for v in G.vertices():  
        V += G.degree(v)  
    return V
```

```
In [4]: """ Returns the degree matrix D of graph G"""  
def deg_matrix(G):  
    return diagonal_matrix([G.degree(v) for v in G.vertices(sort=True)], sparse=False)
```

```
In [5]: """ Returns Ihara Matrix of G """  
  
def ihara_matrix(G):  
    A = G.adjacency_matrix()  
    D = deg_matrix(G)  
    I = identity_matrix(G.order())  
    Z = matrix.zero(G.order())  
    return block_matrix(QQbar, [[A, D-I], [-I, Z]], subdivide=False)
```

```
In [6]: """ Returns the normal transition probability matrix P of graph G """  
  
def trans_prob_matrix(G):  
    A = G.adjacency_matrix()  
    D = diagonal_matrix([G.degree(v) for v in G.vertices()])  
    Dinv = D ^ (-1)  
    P = Dinv * A  
    return P
```

```
In [7]: def edge_ordering_dictionary(G):  
    D = dict()  
    i = 0  
    for u in G.vertices():  
        for v in G.neighbors(u):  
            D.setdefault((u, v), i)  
            i = i+1  
    return D
```

```
In [8]: """ Creates probability matrix of the non-backtracking matrix """
```

```

def trans_prob_matrix_nb(G):
    L = []
    D = edge_ordering_dictionary(G)
    for a in D.keys():
        i = D[a]
        for b in D.keys():
            j = D[b]
            u = a[0]
            v = a[1]
            x = b[0]
            y = b[1]
            if v == x and y != u:
                L.append(float(1)/(G.degree(v)-1))
            else:
                L.append(0)
    P = matrix(vol(G), L)
    return P

```

In [9]: """ Creates normal edge adjacency matrix C for any graph G """

```

def edge_adjacency_matrix(G):
    L = []
    D = edge_ordering_dictionary(G)
    for a in D.keys():
        i = D[a]
        for b in D.keys():
            j = D[b]
            u = a[0]
            v = a[1]
            x = b[0]
            y = b[1]
            if v == x:
                L.append(1)
            else:
                L.append(0)
    B = matrix(vol(G), L)
    return B

```

In [10]: """ Creates non-backtracking matrix B for any graph G """

```

def edge_adjacency_nb_matrix(G):
    L = []
    D = edge_ordering_dictionary(G)
    for a in D.keys():

```

```

i = D[a]
for b in D.keys():
    j = D[b]
    u = a[0]
    v = a[1]
    x = b[0]
    y = b[1]
    if v == x and y != u:
        L.append(1)
    else:
        L.append(0)
B = matrix(vol(G), L)
return B

```

In [11]: """ Uses proposition 5.3 to calculate eigenvalues of Ihara Matrix from A """

```

def compute_K_evals(G):
    K = ihara_matrix(G)
    for A_evec in G.eigenvectors():
        print("lambda: " + str(A_evec[0]))
        for K_evec in K.eigenvectors_right():
            mu = K_evec[0]
            print("Actual mu: " + str(mu))
            G1 = A_evec
            lam = G1[0]
            x_vectors = G1[1]
            for A_evec_same_eval in x_vectors:
                x = list(A_evec_same_eval)
                x = vector(x)
                B_evec = K_evec[1]
                for K_evec_same_eval in B_evec:
                    evec_list = list(K_evec_same_eval)
                    evec_half = evec_list[G.order():]
                    y = vector(evec_half)
                    D = deg_matrix(G)
                    if x*y != 0:
                        mu1_calculated = (
                            lam + sqrt(lam ^ 2 - 4/(x*y)*x*(D - identity_matrix(len(y))*y)) / 2
                        )
                        mu2_calculated = (
                            lam - sqrt(lam ^ 2 - 4/(x*y)*x*(D - identity_matrix(len(y))*y)) / 2
                        )
                        if mu == mu1_calculated or mu == mu2_calculated:
                            print("CORRECT")
                        else:
                            print("INCORRECT")
                    else:

```

```
print("x and y are orthogonal")
print()
```

In [12]: """ Tries to find all eigenvalues of Ihara Matrix using Proposition 5.3. If not all are found, it prints off the graph of G and a list of eigenvalues that weren't calculated. """

Takes a long time

```
def find_eigenvalues(G):
    K = ihara_matrix(G)
    set_of_evals = list(K.eigenvalues())
    for A_evec in G.eigenvectors():
        for K_evec in K.eigenvectors_right():
            mu = K_evec[0]
            lam = A_evec[0]
            x_vectors = A_evec[1]
            for A_evec_same_eval in x_vectors:
                x = list(A_evec_same_eval)
                x = vector(x)
                K_evecors = K_evec[1]
                for K_evec_same_eval in K_evecors:
                    evec_list = list(K_evec_same_eval)
                    evec_half = evec_list[G.order():]
                    y = vector(evec_half)
                    D = deg_matrix(G)
                    if x*y != 0:
                        mu1_calculated = (
                            lam + sqrt(lam ^ 2 - 4/(x*y)*x*(D - identity_matrix(len(y))*y)) / 2
                        )
                        mu2_calculated = (
                            lam - sqrt(lam ^ 2 - 4/(x*y)*x*(D - identity_matrix(len(y))*y)) / 2
                        )
                        if mu == mu1_calculated or mu == mu2_calculated:
                            if mu in set_of_evals:
                                set_of_evals.remove(mu)
    if len(set_of_evals) == 0:
        print("All eigenvalues found")
    else:
        print("These eigenvalues were not found: " + str(set_of_evals))
        show(G)
```

In [13]: """ Returns list of eigenvectors corresponding to jth eigenvalue """

```
def get_ihara_evec(G, j):
    K = ihara_matrix(G)
    K_i = K.eigenvectors_right()[j]
```

```
K_i_evec = K_i[1]
return K_i_evec
```

```
In [14]: """ Tells you if the Ihara matrix is diagonalizable """
```

```
def is_it_diagonalizable(list_of_graphs):
    for g in list_of_graphs:
        k = ihara_matrix(g)
        if k.is_diagonalizable() == False:
            show(g)
```

```
In [15]: """ Checks if two graphs are adjacency cospectral """
```

```
def adjancency_cospectral(g1, g2):
    return g1.spectrum() == g2.spectrum()
```

```
In [16]: """ Checks if two graphs are laplacian cospectral """
```

```
def laplacian_cospectral(g1, g2):
    return g1.kirchhoff_matrix().eigenvalues() == g2.kirchhoff_matrix().eigenvalues()
```

```
In [17]: """ Checks if two graphs are signless cospectral """
```

```
def signless_cospectral(g1, g2):
    return g1.kirchhoff_matrix(signless=True).eigenvalues() == g2.kirchhoff_matrix(signless=True).eigenvalues()
```

```
In [18]: """ Checks if two graphs are normalized laplacian cospectral """
```

```
def normalized_cospectral(g1, g2):
    # TAKES TOO LONG TO RUN THIS
    return g1.kirchhoff_matrix(normalized=True).eigenvalues() == g2.kirchhoff_matrix(normalized=True).eigenvalues()
```

```
In [19]: """ Checks if two graphs are ihara cospectral """
```

```
def ihara_cospectral(g1, g2):
    return ihara_matrix(g1).eigenvalues() == ihara_matrix(g2).eigenvalues()
```

```
In [20]: """ Checks if two graphs are nonbacktracking cospectral """
```

```
def nonbacktracking_cospectral(g1, g2):
    return edge_adjacency_nb_matrix(g1).eigenvalues() == edge_adjacency_nb_matrix(g2).eigenvalues()
```

In [21]: `""" Shows cospectrality in regards to relevant matrices and Ihara matrix """`

```
def show_cospectrality(g1, g2):
    print("Cospectrality")
    print("\tAdjacency: " + str(adjacency_cospectral(g1, g2)))
    print("\tLaplacian: " + str(laplacian_cospectral(g1, g2)))
    print("\tSignless: " + str(signless_cospectral(g1, g2)))
    print("\tIhara: " + str(ihara_cospectral(g1, g2)))
    print("\tNonbacktracking: " + str(nonbacktracking_cospectral(g1, g2)))
```

In [22]:

```
def GM_switch(g1, g2, C):
    if g1.is_regular() != True:
        print("Graph 1 is not regular")
        return
    else:
        J = ones_matrix(g1.order(), g2.order())
        AG = block_matrix(QQbar, [[g1.adjacency_matrix(), C], [
            C.transpose(), g2.adjacency_matrix()]], subdivide=False)
        AH = block_matrix(QQbar, [[g1.adjacency_matrix(
            ), J - C], [J.transpose() - C.transpose(), g2.adjacency_matrix()]], subdivide=False)
        G = Graph(AG)
        H = Graph(AH)
        return G, H
```

In [23]:

```
@parallel(p_iter='reference', ncpus=32)
def graph_list(gen):
    graph_list = [] # Use the correct variable name
    for graph in gen: # Iterate over the generator directly
        # if type(graph) is Graph:
        graph_list.append(graph)
    return graph_list
```

In [24]:

```
@parallel(p_iter='reference', ncpus=32)
def parallel_nb_cospectral_finder(gen): # Now accepts an iterator
    spec_set = set()
    spec_dict = {}

    for G in gen: # Iterate directly over the input
        if type(G) is Graph:
            spectrum = str(edge_adjacency_nb_matrix(G).eigenvalues())
            if spectrum in spec_dict:
```

```

        spec_set.add(spectrum)
        spec_dict[spectrum].append(G)
    else:
        spec_dict.update({spectrum: [G]})

print("About to return")
return spec_set, spec_dict # Return results

```

```

In [25]: def parallel_function(gen):
    spec_set = set()
    spec_dict = {}

    G = next(gen)
    while True:
        try:
            G = next(gen)
            spectrum = str(edge_adjacency_nb_matrix(G).eigenvalues())
            if spectrum in spec_dict:
                spec_set.add(spectrum)
                spec_dict[spectrum].append(G)
            else:
                spec_dict.update({spectrum: [G]})

        except StopIteration:
            break

    for cospectrum in spec_set:
        for G in spec_dict[cospectrum]:
            G.show()

```

```

In [26]: def GM_switch(g1, g2, C):
    if g1.is_regular() != True:
        print("Graph 1 is not regular")
        return
    else:
        J = ones_matrix(g1.order(), g2.order())
        AG = block_matrix(QQbar, [[g1.adjacency_matrix(), C], [
            C.transpose(), g2.adjacency_matrix()]], subdivide=False)
        AH = block_matrix(QQbar, [[g1.adjacency_matrix(
            ), J - C], [J.transpose() - C.transpose(), g2.adjacency_matrix()]], subdivide=False)
        G = Graph(AG)
        H = Graph(AH)
        return G, H

```

```

In [27]: def parse_sage_graphs(graph_strings):
    graph_objects = tuple(Graph(eval(graph_dict))

```

```
        for graph_dict in graph_strings)
    return graph_objects
```

```
In [28]: def all_pairs_ihara_cospectral(graphs):
    n = len(graphs)
    for i in range(n):
        for j in range(i + 1, n):
            if not ihara_cospectral(graphs[i], graphs[j]):
                print('Following graphs are NOT ihara cospectral')
                print(graphs[i].to_dictionary())
                print(graphs[j].to_dictionary())
            else:
                print('Following graphs ARE ihara cospectral')
                print(graphs[i].to_dictionary())
                print(graphs[j].to_dictionary())
```

```
In [29]: def all_pairs_same_ihara_function(graphs):
    n = len(graphs)
    for i in range(n):
        for j in range(i + 1, n):
            if graphs[i].ihara_zeta_function_inverse() != graphs[j].ihara_zeta_function_inverse():
                print('Following graphs DO NOT have same ihara function')
                print(graphs[i].to_dictionary())
                print(graphs[j].to_dictionary())
            else:
                print('Following graphs HAVE same ihara function')
                print(graphs[i].to_dictionary())
                print(graphs[j].to_dictionary())
```

```
In [30]: def read_cospectral_pairs(file_path):
    with open(file_path, 'r') as file:
        lines = file.readlines()
    i = 1
    while i <= len(lines) - 3:
        graph1_str = lines[i].strip()
        graph2_str = lines[i + 1].strip()
        i += 3
        yield (graph1_str, graph2_str)
```

```
In [31]: def plot_eigenvalues_of_matrices(matrices: list):
    colors = rainbow(len(matrices))

    plot = sum(list_plot(matrices[i].eigenvalues(), size=25, color=colors[i], markeredgecolor='black',
                        legend_label=f"Matrix {i} Eigenvalues") for i in range(len(matrices)))
    plot.show(legend_loc='upper right')
```



```
In [32]: from sage.all import CDF
def my_is_similar(A, B):
    A = A.matrix_over_field()
    B = B.matrix_over_field()
    ring = A.base_ring()
    closure = ring.algebraic_closure()
    A = A.change_ring(closure)
    B = B.change_ring(closure)
    _, SA = A.jordan_form(transformation=True)
    _, SB = B.jordan_form(transformation=True)
    SB = SB.change_ring(CDF)
    SB_inverse = SB.inverse()
    return SA, SB_inverse
```

```
In [33]: # Function to normalize each row by its norm squared
def square_normalize_rows(matrix):
    nrows, ncols = matrix.nrows(), matrix.ncols()

    for i in range(nrows):
        # Calculate the norm of the row (Euclidean norm)
        row_norm = sum(matrix[i, j] ^ 2 for j in range(ncols))

        if row_norm != 0: # Avoid division by zero
            for j in range(ncols):
                matrix[i, j] /= row_norm
```

```
In [34]: def reorder_diagonal_matrix(D, right=True):
    # Extract the diagonal elements from the matrix D
    diagonal_elements = [D[i, i] for i in range(D.nrows())]

    # Get the sorted order of the diagonal elements
    sorted_order = sorted(range(len(diagonal_elements)),
                          key=lambda i: diagonal_elements[i])
    # Create a permutation matrix M based on the sorted order
    M = matrix.identity(D.nrows())
    if right:
        M = M[:, sorted_order]
    else:
        M = M[sorted_order, :]

    # Create the new diagonal matrix D' with sorted diagonal elements
    sorted_diagonal_elements = sorted(diagonal_elements)
    D_prime = diagonal_matrix(sorted_diagonal_elements)

    return M, D_prime
```

```
In [35]: def jordan_normal_from_diagonal(D):
    n = D.ncols() # Assuming D is a square matrix
    D_prime = Matrix(QQbar, n, n) # Initialize D' as a zero matrix

    # Copy diagonal entries from D to D'
    for i in range(n):
        D_prime[i, i] = D[i, i]

    # Modify D' to set ones along the superdiagonal for repeated diagonal values
    for i in range(n-1):
        if D_prime[i, i] == D_prime[i+1, i+1]:
            D_prime[i, i+1] = 1

    return D_prime
```

```
In [36]: def similarity_matrix_via_eigenmatrices(A, B):
    DA, SRA = A.right_eigenmatrix()
    # print('DA')
    # print(DA)
    MA, DA_J = reorder_diagonal_matrix(DA)

    DB, SLB = B.left_eigenmatrix()
    # print('DB')
    # print(DB)
    MB, DB_J = reorder_diagonal_matrix(DB, right=False)

    return SRA * MA, MB * SLB
```

```
In [37]: def print_2d_list(dlist):
    print('2d list')
    for row in dlist:
        print(row)
```

```
In [38]: def obtain_matrix_quotient(M):
    rows = int(M.nrows() / 2)
    cols = int(M.ncols() / 2)
    quotient = [[0 for _ in range(cols)] for _ in range(rows)]
    for i in range(rows):
        row = 2*i
        for j in range(cols):
            col = 2*j
            quotient[i][j] = M[row][col] + M[row][col + 1]
    return Matrix(quotient)
```

```
In [39]: def change_matrix_base_ring(matrix, target_ring):  
    for i in range(matrix.nrows()):  
        for j in range(matrix.ncols()):  
            matrix[i,j] = matrix[i,j].real()
```

```
In [40]: def round_matrix_elements(matrix, decimals=3):  
    # Apply rounding element-wise  
    rounded_matrix = matrix.apply_map(lambda x: round(x, decimals))  
    return rounded_matrix
```

```
In [41]: def round_complex_to_real_matrix_elements(matrix, decimals=5, tol=1e-12):  
    # Apply rounding element-wise and handle complex numbers  
    rounded_matrix = matrix.apply_map(lambda x: x.real_part() if abs(x.imag_part()) < tol else x)  
    return rounded_matrix
```

```
In [42]: from fractions import Fraction  
def adjust_repeating_decimals_in_rational_matrix(matrix, limit=1000):  
    """  
    Adjusts the entries of the matrix by converting any decimal that has  
    a large denominator to a rational approximation with a smaller denominator.  
    """  
    for i in range(matrix.nrows()):  
        for j in range(matrix.ncols()):  
            # Get the numerator and denominator of the Sage QQ object  
            num = int(matrix[i,j].numerator())  
            denom = int(matrix[i,j].denominator())  
  
            if denom > limit:  
                # Convert to Python's Fraction object and use limit_denominator  
                fraction = Fraction(num, denom)  
                fraction_approx = fraction.limit_denominator(int(limit))  
  
                # Convert back to Sage QQ and update the matrix  
                matrix[i,j] = QQ(fraction_approx)
```

```
In [43]: def manipulate_similarity_matrix(M):  
    rows = int(M.nrows())  
    cols = int(M.ncols())  
    result = Matrix(QQ, [[0 for _ in range(cols)] for _ in range(rows)])  
    for i in range(0, rows, 2):  
        for j in range(0, cols, 2):  
            a = M[i,j]  
            b = M[i, j + 1]  
            if a == b:  
                result[i,j], result[i+1,j+1], result[i,j + 1], result[i+1,j] = 0,0,0,0  
            else:
```

```

        result[i,j], result[i+1,j+1] = a,a
        result[i,j + 1], result[i+1,j] = b,b
    return result

```

```

In [44]: def subtract_columns_pairs_from_each_other(M):
    rows = int(M.nrows())
    cols = int(M.ncols())
    result = Matrix(QQ, [[0 for _ in range(cols)] for _ in range(rows)])
    for i in range(0, rows, 2):
        for j in range(0, cols, 2):
            a = M[i,j]
            b = M[i, j + 1]
            a,b = b - a, a - b
            result[i,j], result[i+1,j+1] = a,a
            result[i, j + 1], result[i+1,j] = b,b
    return result

```

```

In [45]: def multiply_column_vectors_of_matrix(M):
    ncols = M.ncols()
    for j in range(ncols):
        # Get the column
        col = M.column(j)
        # Find LCM of denominators in the column
        denominators = [x.denominator() for x in col]
        lcm_denominator = lcm(denominators)
        # Multiply the column by its LCM
        col_integer = col * lcm_denominator
        # Replace the column in M
        M.set_column(j, col_integer)

```

```

In [46]: def are_columns_eigenvectors(M):
    for c in range(M.ncols()):
        v = M.column(c)
        try:
            r = M*v/v
        except:
            return False
    return True

```

```

In [47]: # -----
    # 2) Define the 5 possible 2x2 blocks
    # -----
    I_2 = identity_matrix(2)
    J_2 = ones_matrix(2)
    O_2 = Matrix(2)

```

```
A_2 = J_2 * (1/2)
B_2 = J_2 * (-1/2) + I_2
```

```
In [48]: import itertools
from concurrent.futures import ThreadPoolExecutor
from threading import Thread
from sage.all import block_matrix, identity_matrix, ones_matrix, Matrix
from xo_similarity import check_combination

def brute_force_correct_corner_blocks(target_l, target_r):
    """
    Example showing how to use a generator + ProcessPoolExecutor to
    avoid building an enormous list in memory.
    """

    # -----
    # 1) Reduce target_l, target_r to 4x4 block-matrices as you do now
    # -----
    def extract_block(matrix, row_start, col_start):
        return matrix.submatrix(row_start, col_start, 2, 2)

    positions = [(i, j) for i in [0, 8, 10, 18] for j in [0, 8, 10, 18]]
    blocks_l = [extract_block(target_l, row, col) for row, col in positions]
    blocks_r = [extract_block(target_r, row, col) for row, col in positions]
    target_l_small = block_matrix(4, 4, blocks_l)
    target_r_small = block_matrix(4, 4, blocks_r)
    print(f'target_l_small \n')
    print(target_l_small)
    print(f'target_r_small \n')
    print(target_r_small)

    possible_matrices = [A, -A, B, -B, 0]

    # -----
    # 3) Create a generator of all possible 16-block combinations
    #      *without* converting to a huge list
    # -----
    all_combinations = itertools.product(possible_matrices, repeat=16)

    # -----
    # 4) Submit these tasks in parallel using ProcessPoolExecutor
    # -----
    # Using executor.map(...) consumes the generator on-the-fly
    # and returns an iterator of results in the same order.
    with ThreadPoolExecutor() as executor:
        # Map each combination to (combination, target_l_small, target_r_small)
```

```

futures = []
# so check_combination() knows what to do.
step = 1
for combination in all_combinations:
    print(f'combination: {step}')
    f = executor.submit(check_combination, (combination, target_l_small, target_r_small))
    futures.append(f)
    step += 1
for f in futures:
    sim_candidate = f.result()
    if sim_candidate is not None:
        print("Sim candidate found!")
        print(sim_candidate)

```

```

In [50]: def display_all_pairs_of_xo_graphs_and_similarity_matrix(t, graph_pairs_and_sim_matrix):
    for graph_left_and_pos, graph_right_and_pos, similarity_matrix in graph_pairs_and_sim_matrix:
        graph_left, pos_left = graph_left_and_pos
        graph_right, pos_right = graph_right_and_pos

        print('XO PAIR AND SIMILARITY MATRIX')
        A_l : Graph = graph_left.adjacency_matrix()
        A_r : Graph = graph_right.adjacency_matrix()
        D_l = deg_matrix(graph_left)
        D_r = deg_matrix(graph_right)
        # I = Matrix.identity(graph_right.order())
        target_l = ihara_matrix(graph_left)
        target_r = ihara_matrix(graph_right)
        graph_left.show(pos=pos_left)
        print('target_l = ')
        print(target_l)
        graph_right.show(pos=pos_right)
        print('target_r= ')
        print(target_r)

        print('quotient of target_l')
        quotient_l = obtain_matrix_quotient(target_l)
        print(quotient_l)
        print('quotient of target_r')
        quotient_r = obtain_matrix_quotient(target_r)
        print(quotient_r)

        # print('Are they similar?')
        # is_similar = quotient_l.is_similar(quotient_r)
        # print(is_similar)

    my_SA, my_SB = my_is_similar(quotient_l, quotient_r)

```

```

my_S = my_SA * my_SB
my_S = round_complex_to_real_matrix_elements(my_S)
my_S = my_S.change_ring(RDF)
my_S = round_matrix_elements(my_S, 5)
my_S = my_S.change_ring(QQ)
adjust_repeating_decimals_in_rational_matrix(my_S)
# print('my_S')
# print(my_S)

# QLS = quotient_l * my_S
# QSR = my_S * quotient_r
# print(f'QLS == QSR: {QLS == QSR} \n')

# find the inverse of the quotient similarity matrix
my_S_inverse = my_S.inverse()
print('my_S_inverse')
print(my_S_inverse)
continue

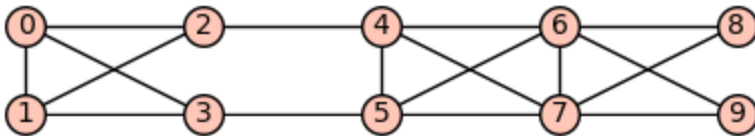
```

```

from xo_similarity import xo_graphs_and_similarity_matrix_gen, laplacian_similarity_matrix_for_xo_graphs_alternating_labels
for n in range(5,8):
    display_all_pairs_of_xo_graphs_and_similarity_matrix(3, xo_graphs_and_similarity_matrix_gen(
        n, laplacian_similarity_matrix_for_xo_graphs_alternating_labels))

```

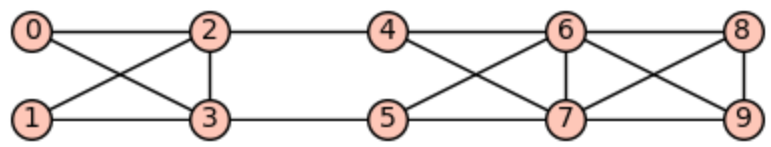
XO PAIR AND SIMILARITY MATRIX



```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  2  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  2  0  0  0  0  0]
[ 0  0  1  0  0  1  1  1  0  0  0  0  0  0  3  0  0  0  0  0]
[ 0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  3  0  0  0  0]
[ 0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  4  0  0  0]
[ 0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  1  0]
[ 0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0]

```




```

target_r=
[ 0  0  1  1  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0]
[ 0  0  1  1  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0]
[ 1  1  0  1  1  0  0  0  0  0  0  0  3  0  0  0  0  0  0]
[ 1  1  1  0  0  1  0  0  0  0  0  0  0  3  0  0  0  0  0]
[ 0  0  1  0  0  0  1  1  0  0  0  0  0  0  2  0  0  0  0]
[ 0  0  0  1  0  0  1  1  0  0  0  0  0  0  0  2  0  0  0]
[ 0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  1  1  0  1  0  0  0  0  0  0  0  0  2  0]
[ 0  0  0  0  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  2]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0]

```

quotient of target_l

```

[ 1  2  0  0  0  2  0  0  0  0]
[ 2  0  1  0  0  0  2  0  0  0]
[ 0  1  1  2  0  0  0  3  0  0]
[ 0  0  2  1  2  0  0  0  4  0]
[ 0  0  0  2  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0]

```

quotient of target_r

```

[ 0  2  0  0  0  1  0  0  0  0]
[ 2  1  1  0  0  0  3  0  0  0]
[ 0  1  0  2  0  0  0  2  0  0]
[ 0  0  2  1  2  0  0  0  4  0]
[ 0  0  0  2  1  0  0  0  0  2]
[-1  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0]

```

my_S_inverse

```

[ 1  0  0  0  0  0  0  0  0  0]
[ 1/2  1  0  0  0  1/2  0  0  0  0]
[ 1/4  0  1  0  0 -1/4  1/2  0  0  0]
[-1/8  1/4  1/2  1  0  1/8 -1/4  3/4  0  0]

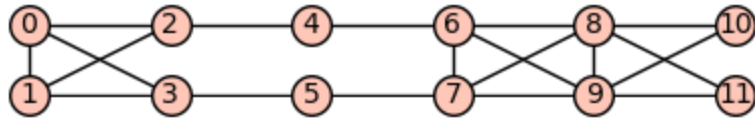
```

```

[ 1/16 -1/8  1/4  1/2  1 -1/16  1/8 -3/8  1  0]
[    0    0    0    0    0    1    0    0    0  0]
[ -1/4    0    0    0    0  1/4  1/2    0    0  0]
[  1/8 -1/4    0    0    0 -1/8  1/4  3/4    0  0]
[-1/16  1/8 -1/4    0    0 1/16 -1/8  3/8  1/2  0]
[ 1/32 -1/16  1/8 -1/4    0 -1/32 1/16 -3/16 1/2  1/2]

```

X0 PAIR AND SIMILARITY MATRIX

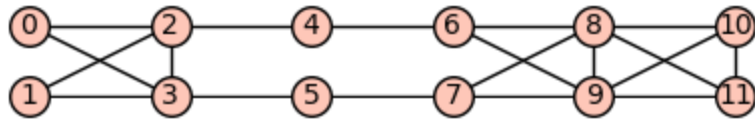


target_l =

```

[ 0  1  1  1  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0]
[ 0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0]
[ 0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  1  0  0  0  0]
[ 0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  3  0  0  0]
[ 0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  3  0  0]
[ 0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  4  0]
[ 0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  4  0]
[ 0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  1]
[ 0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0]

```

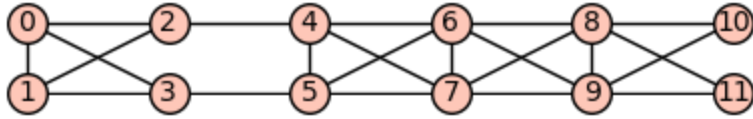



```
[ 0 0 0 0 -1 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0]
```

my_S_inverse

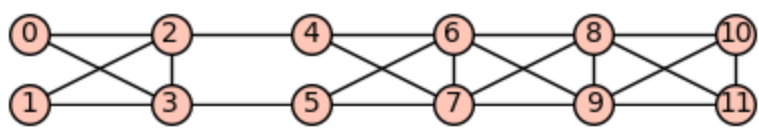
```
[ 1 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 1/2 0 0 0 0 0]
[ 1/4 0 1 0 0 0 -1/4 1/2 0 0 0 0]
[ -1/8 1/4 0 1 0 0 1/8 -1/4 1/4 0 0 0]
[ 1/16 -1/8 1/4 1/2 1 0 -1/16 1/8 -1/8 3/4 0 0]
[ -1/32 1/16 -1/8 1/4 1/2 1 1/32 -1/16 1/16 -3/8 1 0]
[ 0 0 0 0 0 0 1 0 0 0 0 0]
[ -1/4 0 0 0 0 0 1/4 1/2 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 -1/8 1/4 3/4 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 1/16 -1/8 1/8 3/4 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 -1/32 1/16 -1/16 3/8 1/2 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 1/64 -1/32 1/32 -3/16 1/2 1/2]
```

XO PAIR AND SIMILARITY MATRIX



target_l =

```
[ 0 1 1 1 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0]
[ 1 0 1 1 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0]
[ 1 1 0 0 1 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0]
[ 1 1 0 0 0 1 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0]
[ 0 0 1 0 0 1 1 1 0 0 0 0 0 0 0 0 3 0 0 0 0 0 0]
[ 0 0 0 1 1 0 1 1 0 0 0 0 0 0 0 0 0 3 0 0 0 0 0]
[ 0 0 0 0 1 1 0 1 1 1 0 0 0 0 0 0 0 0 4 0 0 0 0]
[ 0 0 0 0 1 1 1 0 1 1 0 0 0 0 0 0 0 0 4 0 0 0 0]
[ 0 0 0 0 0 0 1 1 0 1 1 1 0 0 0 0 0 0 0 4 0 0 0]
[ 0 0 0 0 0 0 0 1 1 1 0 1 1 0 0 0 0 0 0 0 4 0 0]
[ 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0]
[ 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1]
[ -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
```

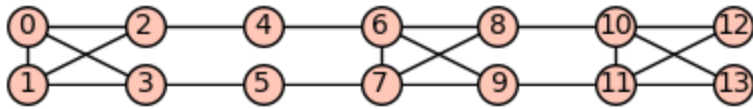



```
[ 0  0  0 -1  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0]
```

my_S_inverse

```
[ 1  0  0  0  0  0  0  0  0  0  0  0]
[ 1/2  1  0  0  0  0  1/2  0  0  0  0  0]
[ 1/4  0  1  0  0  0 -1/4  1/2  0  0  0  0]
[ -1/8  1/4  1/2  1  0  0  1/8 -1/4  3/4  0  0  0]
[ 1/16 -1/8  1/4  1/2  1  0 -1/16  1/8 -3/8  1  0  0]
[ -1/32  1/16 -1/8  1/4  1/2  1  1/32 -1/16  3/16 -1/2  1  0]
[  0  0  0  0  0  0  1  0  0  0  0  0]
[ -1/4  0  0  0  0  0  1/4  1/2  0  0  0  0]
[ 1/8 -1/4  0  0  0  0 -1/8  1/4  3/4  0  0  0]
[ -1/16  1/8 -1/4  0  0  0  1/16 -1/8  3/8  1/2  0  0]
[ 1/32 -1/16  1/8 -1/4  0  0 -1/32  1/16 -3/16  1/2  1/2  0]
[ -1/64  1/32 -1/16  1/8 -1/4  0  1/64 -1/32  3/32 -1/4  1/2  1/2]
```

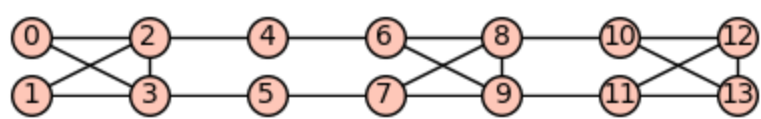
XO PAIR AND SIMILARITY MATRIX



```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0]
[ 0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0]
[ 0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0]
[ 0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0]
[ 0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0]
[ 0  0  0  0  0  0  1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0]
[ 0  0  0  0  0  0  1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0]
[ 0  0  0  0  0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  3  0]
[ 0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  3]
[ 0  0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  1]
[ 0  0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0]

```



target_r=

[0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0]	
[1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0]	
[1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0]	
[0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0]	
[0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0]	
[0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0]	
[0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0]	
[0	0	0	0	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0]	
[0	0	0	0	0	0	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0]	
[0	0	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	2	0	0]	
[0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0]	
[0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2]	
[-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[0	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0]	

quotient of target_l

[1	2	0	0	0	0	0	2	0	0	0	0	0	0]
[2	0	1	0	0	0	0	0	2	0	0	0	0	0]
[0	1	0	1	0	0	0	0	0	1	0	0	0	0]
[0	0	1	1	2	0	0	0	0	0	3	0	0	0]
[0	0	0	2	0	1	0	0	0	0	0	2	0	0]
[0	0	0	0	1	1	2	0	0	0	0	0	3	0]
[0	0	0	0	0	2	0	0	0	0	0	0	0	1]
[-1	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	-1	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	-1	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	-1	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	-1	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	-1	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	-1	0	0	0	0	0	0	0]

quotient of target_r

[0	2	0	0	0	0	0	1	0	0	0	0	0	0]
[2	1	1	0	0	0	0	0	3	0	0	0	0	0]
[0	1	0	1	0	0	0	0	0	1	0	0	0	0]

```

[ 0 0 0 1 0 2 0 0 0 0 0 0 2 0 0 0]
[ 0 0 0 2 1 1 0 0 0 0 0 0 3 0 0]
[ 0 0 0 0 1 0 2 0 0 0 0 0 0 2 0]
[ 0 0 0 0 0 2 1 0 0 0 0 0 0 0 2]
[-1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0]

```

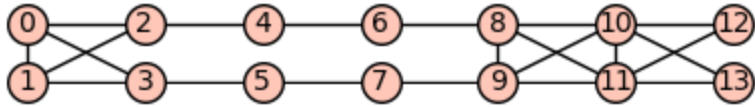
my_S_inverse

```

[ 1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 0 1/2 0 0 0 0 0 0]
[ 1/4 0 1 0 0 0 0 -1/4 1/2 0 0 0 0 0]
[ -1/8 1/4 0 1 0 0 0 1/8 -1/4 1/4 0 0 0 0]
[ 1/16 -1/8 1/4 1/2 1 0 0 -1/16 1/8 -1/8 3/4 0 0 0]
[ -1/32 1/16 -1/8 1/4 0 1 0 1/32 -1/16 1/16 -3/8 1/2 0 0]
[ 1/64 -1/32 1/16 -1/8 1/4 1/2 1 -1/64 1/32 -1/32 3/16 -1/4 3/4 0]
[ 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[ -1/4 0 0 0 0 0 0 1/4 1/2 0 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 0 -1/8 1/4 3/4 0 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 0 1/16 -1/8 1/8 3/4 0 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 0 -1/32 1/16 -1/16 3/8 1/2 0 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 0 1/64 -1/32 1/32 -3/16 1/4 3/4 0]
[ 1/128 -1/64 1/32 -1/16 1/8 -1/4 0 -1/128 1/64 -1/64 3/32 -1/8 3/8 1/2]

```

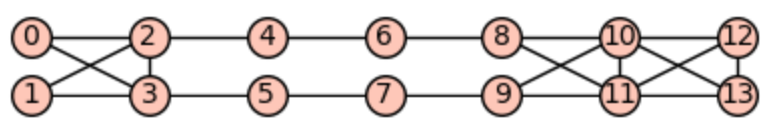
XO PAIR AND SIMILARITY MATRIX



```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0]
[ 0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0]
[ 0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0]
[ 0  0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0]
[ 0  0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0]
[ 0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  3  0  0  0]
[ 0  0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  4  0]
[ 0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  1]
[ 0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]

```



target_r=

[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0]	
[	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0]	
[	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0]	
[	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0]	
[	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0]	
[	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0]	
[	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	2	0	0	0	0]	
[	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0]	
[	0	0	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	4	0]	
[	0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	4	0]	
[	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	2]	
[	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2]	
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0]	

quotient of target_l

[	1	2	0	0	0	0	0	2	0	0	0	0	0	0]
[	2	0	1	0	0	0	0	0	2	0	0	0	0	0]
[	0	1	0	1	0	0	0	0	0	1	0	0	0	0]
[	0	0	1	0	1	0	0	0	0	0	1	0	0	0]
[	0	0	0	1	1	2	0	0	0	0	0	3	0	0]
[	0	0	0	0	2	1	2	0	0	0	0	0	4	0]
[	0	0	0	0	0	2	0	0	0	0	0	0	0	1]
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0]

quotient of target_r

[	0	2	0	0	0	0	0	1	0	0	0	0	0	0]
[	2	1	1	0	0	0	0	0	3	0	0	0	0	0]
[	0	1	0	1	0	0	0	0	0	1	0	0	0	0]

```

[ 0 0 0 1 0 1 0 0 0 0 0 0 1 0 0 0]
[ 0 0 0 0 1 0 2 0 0 0 0 0 2 0 0]
[ 0 0 0 0 2 1 2 0 0 0 0 0 0 4 0]
[ 0 0 0 0 0 2 1 0 0 0 0 0 0 0 2]
[-1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0]

```

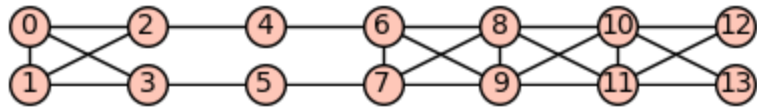
my_S_inverse

```

[ 1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 0 1/2 0 0 0 0 0 0]
[ 1/4 0 1 0 0 0 0 -1/4 1/2 0 0 0 0 0]
[ -1/8 1/4 0 1 0 0 0 1/8 -1/4 1/4 0 0 0 0]
[ 1/16 -1/8 1/4 0 1 0 0 -1/16 1/8 -1/8 1/4 0 0 0]
[ -1/32 1/16 -1/8 1/4 1/2 1 0 1/32 -1/16 1/16 -1/8 3/4 0 0]
[ 1/64 -1/32 1/16 -1/8 1/4 1/2 1 -1/64 1/32 -1/32 1/16 -3/8 1 0]
[ 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[ -1/4 0 0 0 0 0 0 1/4 1/2 0 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 0 -1/8 1/4 3/4 0 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 0 1/16 -1/8 1/8 3/4 0 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 0 -1/32 1/16 -1/16 1/8 3/4 0 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 0 1/64 -1/32 1/32 -1/16 3/8 1/2 0]
[ 1/128 -1/64 1/32 -1/16 1/8 -1/4 0 -1/128 1/64 -1/64 1/32 -3/16 1/2 1/2]

```

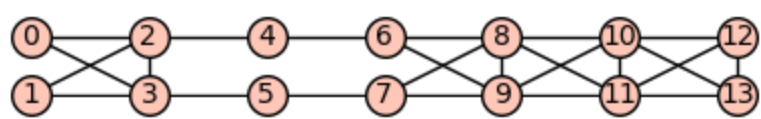
XO PAIR AND SIMILARITY MATRIX



```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0]
[ 0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0]
[ 0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0]
[ 0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0]
[ 0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0]
[ 0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0]
[ 0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  4  0  0  0]
[ 0  0  0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  4  0]
[ 0  0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  1]
[ 0  0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]

```



target_r=

[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0]	
[	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0]	
[	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0]	
[	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0]	
[	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0]	
[	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0]	
[	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0]	
[	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	4	0	0	0]	
[	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	4	0	0]	
[	0	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	4	0]	
[	0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	4]	
[	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	2]	
[	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2]	
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	

quotient of target_l

[	1	2	0	0	0	0	0	2	0	0	0	0	0	0]
[	2	0	1	0	0	0	0	0	2	0	0	0	0	0]
[	0	1	0	1	0	0	0	0	0	1	0	0	0	0]
[	0	0	1	1	2	0	0	0	0	0	3	0	0	0]
[	0	0	0	2	1	2	0	0	0	0	0	4	0	0]
[	0	0	0	0	2	1	2	0	0	0	0	0	4	0]
[	0	0	0	0	0	2	0	0	0	0	0	0	0	1]
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0]

quotient of target_r

[	0	2	0	0	0	0	0	1	0	0	0	0	0	0]
[	2	1	1	0	0	0	0	0	3	0	0	0	0	0]
[	0	1	0	1	0	0	0	0	0	1	0	0	0	0]

```

[ 0 0 0 1 0 2 0 0 0 0 0 0 2 0 0 0]
[ 0 0 0 2 1 2 0 0 0 0 0 0 4 0 0]
[ 0 0 0 0 2 1 2 0 0 0 0 0 0 4 0]
[ 0 0 0 0 0 2 1 0 0 0 0 0 0 0 2]
[-1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0]

```

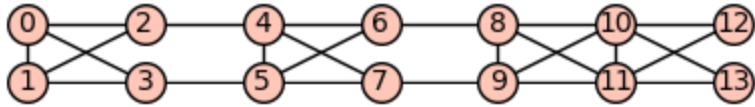
my_S_inverse

```

[ 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 0 1/2 0 0 0 0 0 0 0]
[ 1/4 0 1 0 0 0 0 -1/4 1/2 0 0 0 0 0 0]
[ -1/8 1/4 0 1 0 0 0 1/8 -1/4 1/4 0 0 0 0 0]
[ 1/16 -1/8 1/4 1/2 1 0 0 -1/16 1/8 -1/8 3/4 0 0 0 0]
[ -1/32 1/16 -1/8 1/4 1/2 1 0 1/32 -1/16 1/16 -3/8 1 0 0 0]
[ 1/64 -1/32 1/16 -1/8 1/4 1/2 1 -1/64 1/32 -1/32 3/16 -1/2 1 0 0]
[ 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
[ -1/4 0 0 0 0 0 0 1/4 1/2 0 0 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 0 -1/8 1/4 3/4 0 0 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 0 1/16 -1/8 1/8 3/4 0 0 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 0 -1/32 1/16 -1/16 3/8 1/2 0 0 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 0 1/64 -1/32 1/32 -3/16 1/2 1/2 0 0]
[ 1/128 -1/64 1/32 -1/16 1/8 -1/4 0 -1/128 1/64 -1/64 3/32 -1/4 1/2 1/2 0]

```

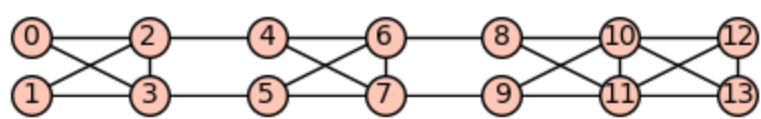
XO PAIR AND SIMILARITY MATRIX




```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0]
[ 0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0  0  0]
[ 0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0  0]
[ 0  0  0  0  1  1  0  0  1  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0]
[ 0  0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0]
[ 0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0]
[ 0  0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  4  0  0  0]
[ 0  0  0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  1  0]
[ 0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0]

```



target_r=

[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0]
[	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0	0]
[	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0]
[	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0]
[	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0	0]
[	0	0	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0]
[	0	0	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0]
[	0	0	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	4	0	0]
[	0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	4	0]
[	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	2	0]
[	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2]
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0]

quotient of target_l

[	1	2	0	0	0	0	0	2	0	0	0	0	0	0]
[	2	0	1	0	0	0	0	0	2	0	0	0	0	0]
[	0	1	1	2	0	0	0	0	0	3	0	0	0	0]
[	0	0	2	0	1	0	0	0	0	0	2	0	0	0]
[	0	0	0	1	1	2	0	0	0	0	0	3	0	0]
[	0	0	0	0	2	1	2	0	0	0	0	0	4	0]
[	0	0	0	0	0	2	0	0	0	0	0	0	0	1]
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0]

quotient of target_r

[	0	2	0	0	0	0	0	1	0	0	0	0	0	0]
[	2	1	1	0	0	0	0	0	3	0	0	0	0	0]
[	0	1	0	2	0	0	0	0	0	2	0	0	0	0]

```

[ 0 0 0 2 1 1 0 0 0 0 0 0 3 0 0 0]
[ 0 0 0 1 0 2 0 0 0 0 0 0 2 0 0]
[ 0 0 0 0 2 1 2 0 0 0 0 0 0 4 0]
[ 0 0 0 0 0 2 1 0 0 0 0 0 0 0 2]
[-1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0]

```

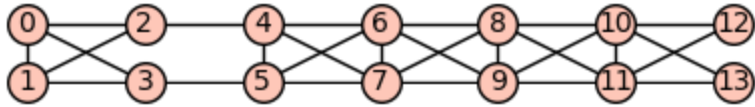
my_S_inverse

```

[ 1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 0 1/2 0 0 0 0 0 0]
[ 1/4 0 1 0 0 0 0 -1/4 1/2 0 0 0 0 0]
[ -1/8 1/4 1/2 1 0 0 0 1/8 -1/4 3/4 0 0 0 0]
[ 1/16 -1/8 1/4 0 1 0 0 -1/16 1/8 -3/8 1/2 0 0 0]
[ -1/32 1/16 -1/8 1/4 1/2 1 0 1/32 -1/16 3/16 -1/4 3/4 0 0]
[ 1/64 -1/32 1/16 -1/8 1/4 1/2 1 -1/64 1/32 -3/32 1/8 -3/8 1 0]
[ 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[ -1/4 0 0 0 0 0 0 1/4 1/2 0 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 0 -1/8 1/4 3/4 0 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 0 1/16 -1/8 3/8 1/2 0 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 0 -1/32 1/16 -3/16 1/4 3/4 0 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 0 1/64 -1/32 3/32 -1/8 3/8 1/2 0]
[ 1/128 -1/64 1/32 -1/16 1/8 -1/4 0 -1/128 1/64 -3/64 1/16 -3/16 1/2 1/2]

```

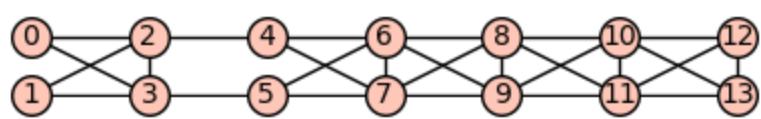
XO PAIR AND SIMILARITY MATRIX



```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0]
[ 0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0  0  0]
[ 0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0  0]
[ 0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0  0  0]
[ 0  0  0  0  1  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0  0]
[ 0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0]
[ 0  0  0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  4  0  0  0]
[ 0  0  0  0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  1  0]
[ 0  0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]

```



target_r=

[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0]	
[	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0	0]	
[	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0	0]	
[	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0]	
[	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0	0	0]	
[	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	4	0	0	0	0	0	0]	
[	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	4	0	0	0	0	0]	
[	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	4	0	0	0	0]	
[	0	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	4	0	0	0]	
[	0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	4	0	0]	
[	0	0	0	0	0	0	0	0	0	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	4	0]	
[	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	2]	
[	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2]	
[	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	
[	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0]	

quotient of target_l

[	1	2	0	0	0	0	2	0	0	0	0	0	0]
[	2	0	1	0	0	0	0	2	0	0	0	0	0]
[	0	1	1	2	0	0	0	0	3	0	0	0	0]
[	0	0	2	1	2	0	0	0	0	4	0	0	0]
[	0	0	0	2	1	2	0	0	0	0	4	0	0]
[	0	0	0	0	2	1	2	0	0	0	0	4	0]
[	0	0	0	0	0	2	0	0	0	0	0	0	1]
[	-1	0	0	0	0	0	0	0	0	0	0	0	0]
[	0	-1	0	0	0	0	0	0	0	0	0	0	0]
[	0	0	-1	0	0	0	0	0	0	0	0	0	0]
[	0	0	0	-1	0	0	0	0	0	0	0	0	0]
[	0	0	0	0	-1	0	0	0	0	0	0	0	0]
[	0	0	0	0	0	-1	0	0	0	0	0	0	0]
[	0	0	0	0	0	0	-1	0	0	0	0	0	0]

quotient of target_r

[	0	2	0	0	0	0	1	0	0	0	0	0	0]
[	2	1	1	0	0	0	0	3	0	0	0	0	0]
[	0	1	0	2	0	0	0	0	2	0	0	0	0]

```

[ 0 0 0 2 1 2 0 0 0 0 0 0 4 0 0 0]
[ 0 0 0 2 1 2 0 0 0 0 0 0 4 0 0]
[ 0 0 0 0 2 1 2 0 0 0 0 0 0 4 0]
[ 0 0 0 0 0 2 1 0 0 0 0 0 0 0 2]
[-1 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0 0]

```

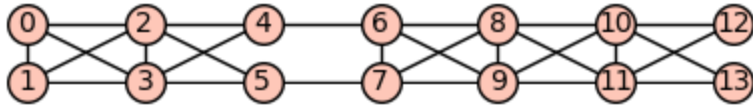
my_S_inverse

```

[ 1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 0 1/2 0 0 0 0 0 0]
[ 1/4 0 1 0 0 0 0 -1/4 1/2 0 0 0 0 0]
[ -1/8 1/4 1/2 1 0 0 0 1/8 -1/4 3/4 0 0 0 0]
[ 1/16 -1/8 1/4 1/2 1 0 0 -1/16 1/8 -3/8 1 0 0 0]
[ -1/32 1/16 -1/8 1/4 1/2 1 0 1/32 -1/16 3/16 -1/2 1 0 0]
[ 1/64 -1/32 1/16 -1/8 1/4 1/2 1 -1/64 1/32 -3/32 1/4 -1/2 1 0]
[ 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[ -1/4 0 0 0 0 0 0 1/4 1/2 0 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 0 -1/8 1/4 3/4 0 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 0 1/16 -1/8 3/8 1/2 0 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 0 -1/32 1/16 -3/16 1/2 1/2 0 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 0 1/64 -1/32 3/32 -1/4 1/2 1/2 0]
[ 1/128 -1/64 1/32 -1/16 1/8 -1/4 0 -1/128 1/64 -3/64 1/8 -1/4 1/2 1/2]

```

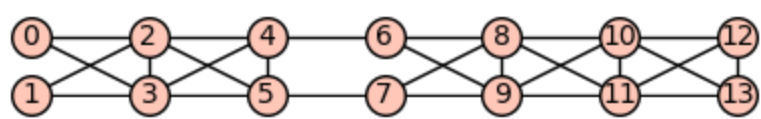
XO PAIR AND SIMILARITY MATRIX



```

target_l =
[ 0  1  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0]
[ 1  0  1  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0]
[ 1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  4  0  0  0  0  0  0  0  0  0  0]
[ 1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0  0  0  0  0  0]
[ 0  0  1  1  0  0  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0]
[ 0  0  1  1  0  0  0  1  0  0  0  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0]
[ 0  0  0  0  1  0  0  1  1  1  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0  0]
[ 0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  3  0  0  0  0  0]
[ 0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  4  0  0  0  0]
[ 0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  4  0  0  0]
[ 0  0  0  0  0  0  0  0  1  1  0  1  1  1  0  0  0  0  0  0  0  0  0  4  0  0]
[ 0  0  0  0  0  0  0  0  0  1  1  1  0  1  1  0  0  0  0  0  0  0  0  0  4  0]
[ 0  0  0  0  0  0  0  0  0  0  1  1  0  1  1  0  0  0  0  0  0  0  0  0  0  1]
[ 0  0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  1]
[-1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0 -1  0  0  0  0  0  0  0  0  0  0  0  0]

```



target_r=

[0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0]
[1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0	0	0	0	0	0	0]
[1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0	0	0	0	0	0]
[0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0	0]
[0	0	1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	3	0	0	0	0	0	0]
[0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0	0]
[0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2	0	0	0	0]
[0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	4	0	0	0]
[0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	4	0	0]
[0	0	0	0	0	0	0	0	1	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	4	0]
[0	0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	4]
[0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	2]
[0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	2]
[-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	0	0	0	0	0	0	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0]

quotient of target_l

[1	2	0	0	0	0	0	2	0	0	0	0	0	0]
[2	1	2	0	0	0	0	0	4	0	0	0	0	0]
[0	2	0	1	0	0	0	0	0	2	0	0	0	0]
[0	0	1	1	2	0	0	0	0	0	3	0	0	0]
[0	0	0	2	1	2	0	0	0	0	0	4	0	0]
[0	0	0	0	2	1	2	0	0	0	0	0	4	0]
[0	0	0	0	0	2	0	0	0	0	0	0	0	1]
[-1	0	0	0	0	0	0	0	0	0	0	0	0	0]
[0	-1	0	0	0	0	0	0	0	0	0	0	0	0]
[0	0	-1	0	0	0	0	0	0	0	0	0	0	0]
[0	0	0	-1	0	0	0	0	0	0	0	0	0	0]
[0	0	0	0	-1	0	0	0	0	0	0	0	0	0]
[0	0	0	0	0	-1	0	0	0	0	0	0	0	0]
[0	0	0	0	0	0	-1	0	0	0	0	0	0	0]

quotient of target_r

[0	2	0	0	0	0	0	1	0	0	0	0	0	0]
[2	1	2	0	0	0	0	0	4	0	0	0	0	0]
[0	2	1	1	0	0	0	0	0	3	0	0	0	0]


```

[ 0 0 0 1 0 2 0 0 0 0 0 2 0 0 0]
[ 0 0 0 2 1 2 0 0 0 0 0 4 0 0]
[ 0 0 0 0 2 1 2 0 0 0 0 0 4 0]
[ 0 0 0 0 0 2 1 0 0 0 0 0 0 2]
[-1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 -1 0 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 -1 0 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 -1 0 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 -1 0 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 -1 0 0 0 0 0 0 0 0]
[ 0 0 0 0 0 0 -1 0 0 0 0 0 0 0]

```

my_S_inverse

```

[ 1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[ 1/2 1 0 0 0 0 0 1/2 0 0 0 0 0 0]
[ 1/4 1/2 1 0 0 0 0 -1/4 1 0 0 0 0 0]
[ -1/8 1/4 0 1 0 0 0 1/8 -1/2 1/2 0 0 0 0]
[ 1/16 -1/8 1/4 1/2 1 0 0 -1/16 1/4 -1/4 3/4 0 0 0]
[ -1/32 1/16 -1/8 1/4 1/2 1 0 1/32 -1/8 1/8 -3/8 1 0 0]
[ 1/64 -1/32 1/16 -1/8 1/4 1/2 1 -1/64 1/16 -1/16 3/16 -1/2 1 0]
[ 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[ -1/4 0 0 0 0 0 0 1/4 1/2 0 0 0 0 0]
[ 1/8 -1/4 0 0 0 0 0 -1/8 1/2 1/2 0 0 0 0]
[ -1/16 1/8 -1/4 0 0 0 0 1/16 -1/4 1/4 3/4 0 0 0]
[ 1/32 -1/16 1/8 -1/4 0 0 0 -1/32 1/8 -1/8 3/8 1/2 0 0]
[ -1/64 1/32 -1/16 1/8 -1/4 0 0 1/64 -1/16 1/16 -3/16 1/2 1/2 0]
[ 1/128 -1/64 1/32 -1/16 1/8 -1/4 0 -1/128 1/32 -1/32 3/32 -1/4 1/2 1/2]

```

In [22]: `from tabnanny import check`

```

def check_invertibility_of_sim_candidate_matrices():
    # Open the file and read its lines.
    with open("xo_ihara_sim_matrix.txt", "r") as f:
        lines = f.readlines()

    matrices = []
    rows = []

    # Process each line.
    for line in lines:

        # Skip lines that say "Sim candidate found!" clear rows list
        if "Sim candidate found!" in line:
            # Create the Sage matrix over QQ.
            matrices.append(Matrix(QQ, rows))
            rows = []
            continue

```

```

# Skip any line that is just a separator (e.g. the dashed line).
if '+' in line:
    continue

# Remove leading/trailing whitespace and remove the square brackets.
line = line.strip().strip("[]")
# Replace the vertical bar with a space so that we have one uniform list of numbers.
line = line.replace("|", " ")
# Split the line into individual number strings.
number_strings = line.split()
# Convert each number string to a rational number in QQ.
row = [QQ(num) for num in number_strings]
# Add the row to our list of rows.
rows.append(row)

# Initialize the list of invertible matrices.
invertible_matrices = []
# Initialize the list of non-invertible matrices.
non_invertible_matrices = []

# Check each matrix for invertibility.
for matrix in matrices:
    if matrix.is_invertible():
        invertible_matrices.append(matrix)
    else:
        non_invertible_matrices.append(matrix)

return invertible_matrices, non_invertible_matrices

```

```

In [23]: invertible_matrices, non_invertible_matrices = check_invertibility_of_sim_candidate_matrices()
print(f'Invertible matrices: {len(invertible_matrices)}')
print(f'Non-invertible matrices: {len(non_invertible_matrices)}')

```

Invertible matrices: 64

Non-invertible matrices: 561

```

In [29]: # Print the invertible matrices to a txt file
with open("invertible_xo_corner_sim_matrices.txt", "w") as f:
    for matrix in invertible_matrices:
        f.write('Invertible similarity corner matrix \n')
        f.write(str(matrix))
        f.write("\n")

```

```

In [ ]: # 1 First trio of ihara cospectral graphs

```

```

# Graph strings
graph_strings = (

```

```

    "{0: [3, 5, 7, 9], 1: [4, 6, 7, 9], 2: [5, 6, 8, 9], 3: [0, 7, 8, 9], 4: [1, 6, 8, 9], 5: [0, 2, 8, 9], 6: [1, 2, 4, 7, 9], 7: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]},
    "{0: [3, 5, 7, 9], 1: [4, 6, 7, 9], 2: [5, 6, 8, 9], 3: [0, 7, 8], 4: [1, 6, 8, 9], 5: [0, 2, 8, 9], 6: [1, 2, 4, 7, 9], 7: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]},
    "{0: [4, 6, 7, 9], 1: [4, 6, 8, 9], 2: [5, 6, 7, 9], 3: [5, 7, 8], 4: [0, 1, 8, 9], 5: [2, 3, 8, 9], 6: [0, 1, 2, 7, 9], 7: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]}
)

# Parse and convert to Sage graph objects
graphs = parse_sage_graphs(graph_strings)

# Double check if same ihara function
all_pairs_same_ihara_function(graphs)

all_pairs_ihara_cospectral(graphs)

for g in graphs:
    show(g)

```

```

In [ ]: # 2 Second trio of ihara cospectral graphs

# Graph strings
graph_strings = (
    "{0: [3, 6, 7, 9], 1: [4, 5, 6, 8], 2: [4, 5, 7, 9], 3: [0, 6, 8, 9], 4: [1, 2, 7, 8, 9], 5: [1, 2, 8, 9], 6: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]},
    "{0: [3, 4, 6, 7], 1: [4, 5, 8, 9], 2: [5, 6, 8, 9], 3: [0, 6, 7, 9], 4: [0, 1, 7, 8, 9], 5: [1, 2, 7, 9], 6: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]},
    "{0: [3, 4, 6, 7], 1: [4, 5, 8, 9], 2: [5, 6, 7, 9], 3: [0, 6, 8, 9], 4: [0, 1, 7, 8, 9], 5: [1, 2, 7, 9], 6: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]}
)

# Parse and convert to Sage graph objects
graphs = parse_sage_graphs(graph_strings)

# Double check if same ihara function
all_pairs_same_ihara_function(graphs)

all_pairs_ihara_cospectral(graphs)

for g in graphs:
    show(g)

```

```

In [ ]: # This cell test all pairs of graphs for the given parameters below
# Parameters:
m = 6
d = 3
k = 4
C = matrix([[0, 1, 0, 0, 1, 1], [1, 1, 1, 0, 0, 0],
            [0, 0, 1, 1, 0, 1], [1, 0, 0, 1, 1, 0]])
C_t = C.transpose()
print(C_t)
g1_graphs = graphs.nauty_geng(str(m) + " -c -d" + str(d) + " -D" + str(d))
g1_graphs_list = list(g1_graphs)

```

```

g2_graphs = graphs.nauty_geng(str(k) + "-c -d2")
g2_graphs_list = list(g2_graphs)

for g1 in g1_graphs_list:
    for g2 in g2_graphs_list:
        G1, H = GM_switch(g1, g2, C_t)
        if not ihara_cospectral(G1, H):
            show(g1)
            show(g2)
            show(G1)
            show(H)
            print("Not K cospectral")

```

In []: *# Generated all-cospectral graphs using two graphs on 6 vertices*

```

C = matrix([[1, 1, 1, 0, 0, 0], [1, 0, 1, 1, 0, 0], [1, 0, 0, 1, 1, 0], [
            0, 1, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1], [0, 1, 0, 0, 1, 1]])
g1 = Graph({0: [1, 5], 4: [5, 3], 2: [1, 3]})
g2 = Graph({5: [2, 4], 3: [2, 4], 1: [0, 2]})
list = GM_switch(g1, g2, C)
for G1 in list:
    show(G1)

g1 = list[0]
g2 = list[1]

print("Are isomorphic?: " + str(g1.is_isomorphic(g2)))
show_cospectrality(g1, g2)

```

In []: *# Generated all-cospectral graphs using two graphs on 4 vertices. Unfortunately, they are isomorphic*

```

C = matrix([[1, 1, 0, 0], [1, 0, 1, 0], [0, 1, 0, 1], [0, 0, 1, 1]])

g1 = Graph({0: [1, 3], 2: [1, 3]})
g2 = Graph({0: [1, 2], 2: [1, 3]})

list = GM_switch(g1, g2, C)
for G1 in list:
    show(G1)

g1 = list[0]
g2 = list[1]

print(g1.is_isomorphic(g2))

```

```

# are_cospectral(graphs[0],graphs[1])

print("these are the adjacency matrix spectrums: ")
print(g1.spectrum())
print(g2.spectrum())
print()

print("these are the laplacian spectrums: ")
print(g1.kirchhoff_matrix().eigenvalues())
print(g2.kirchhoff_matrix().eigenvalues())
print()

print("these are the normalized laplacian spectrums: ")
print(g1.kirchhoff_matrix(normalized=True).eigenvalues())
print(g2.kirchhoff_matrix(normalized=True).eigenvalues())
print()

print("these are the signless laplacian spectrums: ")
print(g1.kirchhoff_matrix(signless=True).eigenvalues())
print(g2.kirchhoff_matrix(signless=True).eigenvalues())
print()

print("these are the ihara matrix spectrums: ")
print(ihara_matrix(g1).eigenvalues())
print(ihara_matrix(g2).eigenvalues())

```

In []: *# Generated all-cospectral graphs using two graphs on 8 and 6 vertices*

```

C = matrix([[1, 1, 1, 0, 0, 0], [0, 1, 1, 1, 0, 0], [0, 0, 1, 1, 1, 0], [0, 0, 0, 1, 1, 1], [
    1, 0, 0, 0, 1, 1], [1, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 1], [0, 1, 0, 1, 1, 0]])
g1 = Graph({0: [1, 7], 2: [1, 3], 4: [3, 5], 6: [5, 7]})
g2 = Graph({0: [1, 2, 3, 4, 5], 3: [1, 2], 4: [5]})

list = GM_switch(g1, g2, C)
for G1 in list:
    show(G1)

g1 = list[0]
g2 = list[1]

print("Isomorphic? " + str(g1.is_isomorphic(g2)))

```

```
# are_cospectral(graphs[0],graphs[1])
```

```
show_cospectrality(g1, g2)
```

```
In [ ]: # Testing cospectrality of graphs on minimum 2 degrees
```

```
all_spectrums = []
all_graphs = list(list.nauty_geng("9 -d2"))
for g in all_graphs:
    k_g = ihara_matrix(g)
    all_spectrums.append(k_g.eigenvalues())

graphs_length = len(all_spectrums)
for i in range(0, graphs_length):
    for j in range(0, graphs_length):
        if i != j and all_spectrums[i] == all_spectrums[j]:
            show(all_graphs[i])
            show(all_graphs[j])
```

```
In [ ]: # IMPORTANT: FIND MORE COUNTER EXAMPLES AND TEST HYPOTHESIS
```

```
# Counter example is only cospectral in regards to signless laplacian and ihara matrix.
# We later show that cospectrality in terms of signless laplacian does not imply cospectrality in terms of ihara matrix

# Counter example to our 'conjecture' about cospectrality of B or K matrices

g1 = {0: [3, 6, 7], 6: [3, 1], 7: [3, 1, 5, 4], 4: [1, 5, 2], 5: [1, 2]}
g1 = Graph(g1)
show(g1)

g2 = {0: [3, 4, 6, 7], 3: [4, 6, 7], 6: [4, 1], 5: [1, 2, 7], 7: [1, 2]}
g2 = Graph(g2)
show(g2)

show_cospectrality(g1, g2)
```

```
In [ ]: # FINDINGS: being cospectral in regards to signless laplacian does NOT imply cospectrality in regards to Ihara matrix
```

```
# Two graphs on 10 vertices which are cospectral in terms of Q (signless laplacian) matrix

g1 = {0: [1, 4, 5, 9], 6: [1, 5, 7], 2: [1, 3, 7], 8: [7, 9], 4: [3, 9]}
g2 = {0: [1, 4, 5], 6: [1, 5, 7], 2: [1, 3, 7], 8: [7, 9], 4: [3, 5, 9]}

g1 = Graph(g1)
g2 = Graph(g2)

show(g1)
```

```
show(g2)

show_cospectrality(g1, g2)
```

```
In [ ]: # Another counter-example like the previous

g1 = {0: [4], 1: [0, 2, 3, 6, 8], 3: [2, 4], 5: [6, 9], 7: [6, 8], 9: [8]}
g2 = {0: [4], 1: [0, 2, 3, 6], 3: [2, 4, 6], 5: [6, 9], 7: [6, 8], 9: [8]}

g1 = Graph(g1)
g2 = Graph(g2)

show(g1)
show(g2)

show_cospectrality(g1, g2)
```

```
In [ ]: # ANOTHER NON-REGULAR GRAPH WHICH IS COSPECTRAL IN REGARDS TO IHARA MATRIX WITH MIN DEGREE 2

# Example from paper Dr. Kempton sent us

g1 = {0: [1, 5, 8, 9], 1: [4, 5, 6], 2: [3, 6, 7, 9], 3: [4, 7, 8], 5: [4, 6]}
g2 = {0: [1, 4, 6, 7], 1: [7, 8, 9], 2: [4, 5, 8, 3], 3: [5, 6, 9], 5: [4, 6]}

g1 = Graph(g1)
g2 = Graph(g2)

show(g1)
show(g2)

show_cospectrality(g1, g2)
```

```
In [ ]: # Testing diagonalizability of Ihara Matrix of graphs with 8 vertices, connected, and d_min >= 2

all_graphs = list.nauty_geng("9 -c -d2")
print(len(list(all_graphs)))
# for g in all_graphs:
#     find_eigenvalues(g)
# is_it_diagonalizable(all_graphs)
```

```
In [ ]: # example of two cospectral graphs with minimum degree at least 2

g1 = {0: [1, 7, 8], 1: [8, 2], 3: [2, 4], 5: [4, 6], 7: [6, 8], 8: [4]}
g1 = Graph(g1)
show(g1)
```

```

g2 = {0: [1, 7, 8], 1: [8, 2], 3: [2, 8, 4], 5: [4, 6], 7: [6], 8: [4]}
g2 = Graph(g2)
show(g2)
show_cospectrality(g1, g2)
k_g1 = ihara_matrix(g1)
k_g2 = ihara_matrix(g2)

print(k_g1.charpoly())
print(k_g2.charpoly())

```

In []: *# reconstructing the counter-example to our conjecture about the diagonalizability of Ihara matrices*

```

g = {0: [3, 6, 5], 1: [4, 5, 6], 2: [4, 5, 6], 3: [0, 6],
     4: [1, 2, 5], 5: [0, 1, 2, 4, 6], 6: [0, 3, 5]}
sad_graph = Graph(g)
show(sad_graph)
k_sad = ihara_matrix(sad_graph)
print("Is diagonalizable: " + str(k_sad.is_diagonalizable()))
print(k_sad.eigenvalues())
find_eigenvalues(sad_graph)

```

In []: *# Testing several different graphs to see if they preserve their property of having an equitable partition*

```

# G = Graph({})
# G.add_cycle([0,1,2,3])
# G.add_cycle([4,5,6,7])
# G.add_edges([(0,4),(1,5),(2,6),(3,7)])

G1 = list.CompleteBipartiteGraph(3, 3)

G1 = DiGraph(edge_adjacency_nb_matrix(G1))

print(G1.is_equitable([[0, 1], [2], [3, 4]]))

# G = Graph({7:[0,1,3,5,6], 1:[0,2,3,5], 3:[2,4,5], 5:[4,6]})

# G = Graph({0:[1,2,3,6], 7:[1,4,5,6], 1:[2,4], 6:[3,5], 2:[3,4], 5:[3,4]})

# G = Graph({2:[0,1,3,4], 3:[1]})

show(G1)
# print(G.adjacency_matrix())
# print()
# print(edge_adjacency_nb_matrix(G))

```



```
In [ ]: # checking if nonbacktracking graph has an equitable partition
```

```
G1 = Graph({2: [0, 1, 3, 4], 0: [1]})  
# print(edge_adjacency_nb_matrix(G))  
G1 = DiGraph(edge_adjacency_nb_matrix(G1))  
show(G1)  
print(G1.coarsest_equitable_refinement(  
    [[0, 2], [1, 3], [4, 5], [6, 7], [8, 9]]))  
# G.is_equitable([[0,1],[2],[3,4]])  
# G.is_equitable(P, quotient_matrix=True)
```

```
In [ ]: # An example of proposition 5.3 calculating the eigenvalues of the Ihara Matrix of some graph on 9 vertices
```

```
G1 = deg_two_graph(6, 0.5)  
G1.add_vertices([6, 7, 8])  
G1.add_edges([(6, 0), (7, 0), (8, 7)])  
show(G1)  
K = ihara_matrix(G1)  
print("Ihara matrix is diagonalizable: " + str(K.is_diagonalizable()))  
print(G1.adjacency_matrix().eigenvalues())  
compute_K_evals(G1)
```

```
In [ ]: # testing cospectral 3-regular graphs on 16 vertices
```

```
g1 = {0: [1, 3, 7], 2: [3, 1, 4], 5: [4, 6, 7], 6: [4, 8], 9: [8, 7, 11], 10: [3, 8, 12], 11: [1, 15], 13: [12, 14, 15], 14: [12, 15], }  
g2 = {0: [1, 6, 9], 2: [1, 3, 4], 3: [4, 11], 5: [4, 6, 7], 8: [6, 7, 10], 9: [11, 12], 10: [1, 15], 11: [7], 13: [12, 14, 15], 14: [12, 15]}  
g1 = Graph(g1)  
g2 = Graph(g2)  
show(g1)  
show(g2)  
show_cospectrality(g1, g2)
```

```
In [ ]: # using cospectral_graphs() built in function to find cospectral graphs
```

```
gen = list.nauty_geng("12 -d3 -D3")  
len(list(gen))  
# g = graphs.cospectral_graphs(12, graphs=graphs.nauty_geng(12, -d3, -D3))
```

```
In [ ]: g1 = {0: [2, 3, 4, 5], 1: [2, 3, 4, 5], 2: [3, 6], 3: [4], 5: [4, 7]}  
g1 = Graph(g1)  
show(g1)  
g2 = {0: [2, 3, 4, 5], 1: [2, 3, 4, 5], 2: [3, 6], 3: [4], 5: [4, 7], 6: [7]}  
g2 = Graph(g2)  
show(g2)  
print(g1.spectrum())
```

```
print(g2.spectrum())  
show_cospectrality(g1, g2)
```

```
In [ ]: # These are two really big bi-regular graphs that are supposed to be cospectral. I screwed it up though  
  
g1 = {0: [1, 5, 15], 1: [2, 10], 2: [3, 7], 3: [4, 12], 4: [5, 9], 6: [5, 11, 7], 8: [7, 13, 9], 10: [9, 11], 12: [11,  
      26, 49], 26: [27, 50], 27: [28, 53], 28: [29, 34], 29: [30, 35], 30: [31, 36], 31: [32, 37], 32: [33, 38], 33: [47,  
g1 = Graph(g1)  
g2 = {0: [1, 5, 33], 1: [2, 10], 2: [3, 7], 3: [4, 12], 4: [5, 9], 6: [5, 11, 7], 8: [7, 13, 9], 10: [9, 11], 12: [11,  
      25, 50], 25: [26, 51], 26: [27, 52], 27: [28, 33], 28: [29, 34], 29: [30, 35], 30: [31, 36], 31: [32, 37], 32: [38]  
g2 = Graph(g2)  
print(g1.adjacency_matrix().eigenvalues() ==  
      g2.adjacency_matrix().eigenvalues())
```